



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U03_A05 Casos prácticos, Ponte a prueba, etc... -
Prof. Emilio Ramírez**

Autor: Segura Abad, Mario

Fecha: 28 de Noviembre, 2025

ÍNDICE

INTRODUCCIÓN..... 3

ENUNCIADO 4

PROBAD TODO EL CÓDIGO DEL APARTADO 54

CASO PRÁCTICO 5.....5

Filtrado de datos con expresiones regulares5

PONTE A PRUEBA 56

Validación de un documento de identidad6

 CLAVES DE RESOLUCIÓN:.....7

 • *Divide el problema en cuatro partes: el primer carácter, los siete siguientes, el carácter 9 y el último.*.....7

 • *Recuerda que los corchetes [] permiten definir caracteres y rangos de los que elegir ([LM] hace referencia al carácter L o al carácter M).*7

DESARROLLO DE LA PRÁCTICA 8

 DESCRIPCIÓN8

 1. *Una función que extrae etiquetas alfanuméricas desde una cadena con separadores variados.*.....8

 2. *Una expresión regular que valida un documento de identidad ficticio con reglas específicas. Se ha trabajado con el archivo DWEC_U03_A05_CP_E.js y con ejemplos de validación en scripts JavaScript.*8

 CONTENIDO DEL ARCHIVO.....8

Código del Apartado 5 de teoría.....8

Caso Práctico 5 – Filtrado de datos con expresiones regulares9

Ponte a Prueba 5 – Validación de un documento de identidad.....9

 CÓMO VISUALIZARLO10

Requisitos:10

Pasos:.....10

 OBJETIVOS DE LA PRÁCTICA.....11

CONCLUSIÓN..... 12

INTRODUCCIÓN

En el ejercicio “Probad todo el código del Apartado 5” deberemos incluir en uno o varios fuentes (siempre recomendando en varios) absolutamente todo el código que aparece en la teoría del apartado 5 de JavaScript.

En el “Caso práctico 5” se debe realizar una función que reciba una cadena con una lista de etiquetas y nos devuelva un *array* con las etiquetas que estén en la propia lista. Para ello, disponemos del archivo DWEC_U03_A05_CP_E.js con ejemplos de uso de la función en cuestión.

En el “Ponte a prueba 5” nos realizan la siguiente pregunta: ¿Qué expresión regular utilizarías para comprobar documento de identidad ficticio de 10 caracteres con las siguientes reglas? Asimismo, deberemos crear en JS varios ejemplos de documentos de identidad tanto válidos como no válidos y escribir el código para probar su correcto funcionamiento.

ENUNCIADO

Probad todo el código del Apartado 5

Incluid en uno o varios fuentes **todo** el código que aparece en la teoría de este apartado, así como su ejecución y prueba de autoría.

Ejemplo de implementación:

Captura de pantalla – DWECE_U03_A05_TEO_1.js

```
JS PonteAPrueba5.js JS DWECE_U03_A05_TEO_1.js X
JS DWECE_U03_A05_TEO_1.js > [e] cadena1
1 // Autor: Mario Segura Abad
2 // Fecha: 25/11/2025
3
4 var cadena1 = "Hola mundo! Hola de nuevo! Probaremos diferentes cadenas,"
5 "como AAbCC o rrXbCC o i98AA34bCC";
6
7 /hola/ . test(cadena1); // false
8 /hola/i . test(cadena1); // true (bandera 'i')
9
10 cadena1.replace(/Hola/, "Hello"); // 'Hello mundo! Hola de nuevo! ...'
11 cadena1.replace(/Hola/g, "Hello"); // 'Hello mundo! Hello de nuevo! ...'
12
13 cadena1.match(/bCC/); // Array cuya primera posición es 'bCC'
14 cadena1.match(/bCC/g); // [ 'bCC', 'bCC', 'bCC' ]
15
16 cadena1.match(/\w*!/); // Array cuya primera posición es 'mundo!'
17 cadena1.match(/\w*!/g); // [ 'mundo!', 'nuevo!' ]
18
19 cadena1.match(/\w*bCC/); // Array cuya primera posición es 'AAbCC'
20 cadena1.match(/\w*bCC/g); // [ 'AAbCC', 'rrXbCC', 'i98AA34bCC' ]
21 cadena1.replace(/\w*bCC/g, "c"); // 'Hola mundo! Hola de nuevo! Probaremos diferentes cadenas, como c o c o c'
```

Captura de pantalla – DWECE_U03_A05_TEO_2.js

```
JS PonteAPrueba5.js JS DWECE_U03_A05_TEO_2.js X
JS DWECE_U03_A05_TEO_2.js > ...
1 // Autor: Mario Segura Abad
2 // Fecha: 25/11/2025
3
4
5 // Filtrar números de teléfono
6 var tel1 = "612-443-355";
7 var tel2 = "612 44 33 55";
8 var tel3 = "(612) 44 33 55";
9 var regTel = /\d/g; // Se buscan caracteres numéricos
10
11 // 'match' devuelve un array con las coincidencias; 'join' combina los elementos de un array
12 // con el delimitador indicado
13 tel1.match(regTel).join(""); // '612443355'
14 tel2.match(regTel).join(""); // '612443355'
15 tel3.match(regTel).join(""); // '612443355'
16
17 // Otra opción con 'replace'. '\d' hace referencia a dígitos no numéricos
18 // Se reemplazan los dígitos no numéricos con el carácter vacío
19 var regTel2 = /\D/g; // Se buscan caracteres numéricos
20 tel1.replace(regTel2, ""); // '612443355'
21 tel2.replace(regTel2, ""); // '612443355'
22 tel3.replace(regTel2, ""); // '612443355'
```

Caso práctico 5

Filtrado de datos con expresiones regulares

Se desea realizar una función que reciba una cadena con un listado de etiquetas y devuelva un *array* con las etiquetas que haya presentes.

Las etiquetas estarán formadas únicamente por caracteres alfanuméricos (letras y números). Para dar facilidades a los usuarios, podrán estar separadas de diferentes maneras: ,, ., :, ;, espacio en blanco, etcétera.

Adjunto dispones del archivo **DWEC_U03_A05_CP_E.js**, con ejemplos de uso de la función.

Ejemplo de implementación:

Captura de pantalla – DWEC_U03_A05_CP_E.js

```
JS DWEC_U03_A05_z_CP_E.js ×
CasoPractico > JS DWEC_U03_A05_z_CP_E.js > ...
1 // Autor: Mario Segura Abad
2 // Fecha: 25/11/2025
3
4 function obtenerEtiquetas(cadena) {
5     // Buscar todas las secuencias alfanuméricas
6     return cadena.match(/[A-Za-z0-9]+/g) || [];
7 }
8
9 // Ejemplos de uso
10 obtenerEtiquetas("eti1, eti2, eti3"); // ["eti1", "eti2", "eti3"]
11 obtenerEtiquetas("eti1 eti2 eti3"); // ["eti1", "eti2", "eti3"]
12 obtenerEtiquetas("eti1:eti2:eti3"); // ["eti1", "eti2", "eti3"]
13 obtenerEtiquetas("eti1, eti2 ; eti3"); // ["eti1", "eti2", "eti3"]
14 obtenerEtiquetas("eti1.eti2 ,eti3"); // ["eti1", "eti2", "eti3"]
```

Ponte a prueba 5

Validación de un documento de identidad

¿Qué expresión regular utilizarías para comprobar un documento de identidad ficticio de 10 caracteres con las siguientes reglas?

- Carácter 1: un número entre 0 y 9 o una letra **A**, **B** o **C**.
- Caracteres 2 a 8: números entre 0 y 9.
- Carácter 9: una letra **X**, **Y** o **Z**.
- Carácter 10: un número entre 1 y 5.

Crea en JavaScript varios ejemplos de documentos de identidad válidos y no válidos de ejemplo y escribe el código para testear el correcto funcionamiento de la expresión.

Ejemplo de implementación:

Captura de pantalla – PonteAPrueba5.js

```
JS PonteAPrueba5.js ×
PonteAPrueba > JS PonteAPrueba5.js > [🔗] ejemplosValidos
1 // Autor: Mario Segura Abad
2 // Fecha: 25/11/2025
3
4
5 // Expresión regular del documento
6 const regexDoc = /^[0-9ABC][0-9]{7}[XYZ][1-5]$/;
7
8 // Ejemplos válidos
9 let ejemplosValidos = [
10   "A1234567X3",
11   "01234567Y5",
12   "C9999999Z1",
13   "34567890X2"
14 ];
15
16 // Ejemplos no válidos
17 let ejemplosInvalidos = [
18   "D1234567X3", // Carácter 1 no permitido
19   "A12345X7X3", // Caracteres 2-8 incorrectos
20   "A1234567W3", // Carácter 9 no permitido
21   "A1234567X8", // Carácter 10 fuera de rango
22   "A1234567X33", // Demasiados caracteres
23   "A123456" // Muy corto
24 ];
25
26 // Test de validación
27 console.log("---- DOCUMENTOS VÁLIDOS ----");
28 ejemplosValidos.forEach(doc => {
29   console.log(doc, regexDoc.test(doc));
30 });
```

Claves de resolución:

- Divide el problema en cuatro partes: el primer carácter, los siete siguientes, el carácter 9 y el último.
- Recuerda que los corchetes `[]` permiten definir caracteres y rangos de los que elegir (`[LM]` hace referencia al carácter **L** o al carácter **M**).

DESARROLLO DE LA PRÁCTICA

Práctica DWEK – Unidad 3 Apartado 5

Descripción

Este apartado se centra en el uso de **expresiones regulares en JavaScript** para filtrar y validar datos. El objetivo principal ha sido implementar:

1. Una función que extrae etiquetas alfanuméricas desde una cadena con separadores variados.
2. Una expresión regular que valida un documento de identidad ficticio con reglas específicas. Se ha trabajado con el archivo DWEK_U03_A05_CP_E.js y con ejemplos de validación en scripts JavaScript.

Contenido del archivo

Código del Apartado 5 de teoría

Se ha probado todo el código relacionado con:

- Creación de patrones con clases de caracteres y rangos ([A-Za-z0-9], [0-9], [A-Ca-c], [X-Zx-z]).
- Uso de anclajes (^, \$) y cuantificadores ({n}, +).
- Métodos de JavaScript para trabajar con regex: match, test, replace.

Cada archivo incluye comentarios con el nombre del autor y la fecha de realización. Se han capturado los resultados por consola como prueba de ejecución.

Ejemplo de ejecución:

Salida por consola – DWEK_U03_A05_TEO_1.js

```

  Autor: Mario Segura Abad
  Fecha de realización: 25/11/2025
  . 'Hola mundo! Hola de nuevo! Probaremos diferentes cadenas,'
  |

```

Salida por consola – DWEK_U03_A05_TEO_2.js

```

  testRegex(regex2, '612443355', // 012443355
  Autor: Mario Segura Abad
  Fecha de realización: 25/11/2025
  < '612443355'

```


Caso Práctico 5 – Filtrado de datos con expresiones regulares

1. Función de filtrado de etiquetas

- Recibe una cadena con etiquetas separadas por distintos símbolos (coma, punto, dos puntos, punto y coma, espacios, etc.).
- Devuelve un array con las etiquetas presentes, formadas únicamente por caracteres alfanuméricos.
- Se ha utilizado el patrón `[A-Za-z0-9]+` con flag `g`.

2. Ejemplo de uso

- Entrada: “web, backend; api: seguridad php7.node”
- Salida: [“web”, “backend”, “api”, “seguridad”, “php7”, “node”]

Este ejercicio ha reforzado el uso de expresiones regulares para **extraer datos de cadenas con separadores heterogéneos**.

Ejemplo de ejecución:

Salida por consola – DWEC_U03_A05_CP_E.js

```
const etiquetas = "web, backend; api: seguridad php7.node";
// ...
Autor: Mario Segura Abad
Fecha de realización: 25/11/2025
< (3) ['eti1', 'eti2', 'eti3'] i
  0: "eti1"
  1: "eti2"
  2: "eti3"
  length: 3
  ▶ [[Prototype]]: Array(0)
>
```

Ponte a Prueba 5 – Validación de un documento de identidad

1. Formato del documento (10 caracteres)

- Carácter 1: número 0-9 o letra A, B o C (mayúscula o minúscula).
- Caracteres 2-8: números 0-9.
- Carácter 9: letra X, Y, Z (mayúscula o minúscula).
- Carácter 10: número entre 1 y 5.

2. Expresión regular utilizada

`^([0-9]|[A-Ca-c])[0-9]{7}([X-Zx-z])[1-5]$`

Ejemplos válidos:

- A1234567Z3
- 72345678y5
- c0000000X1

Ejemplos no válidos:

- D1234567Z3 (D no permitida)
- A123456Z3 (faltan dígitos)
- A1234567Q3 (Q no permitida)
- A1234567Z0 (0 no permitido al final)
- A1234567Z6 (6 fuera de rango)

Este ejercicio ha reforzado el uso de **regex para validaciones estrictas de formato**.

Ejemplo de ejecución:

Salida por consola – PonteAPrueba5.js

```
----- DOCUMENTOS VÁLIDOS -----
A1234567X3 true
01234567Y5 true
C9999999Z1 true
34567890X2 true
----- DOCUMENTOS INVÁLIDOS -----
D1234567X3 false
A12345X7X3 false
A1234567W3 false
A1234567X8 false
A1234567X33 false
A123456 false
Autor: Mario Segura Abad
Fecha de realización: 25/11/2025
```

Cómo visualizarlo

Requisitos:

- Editor de código compatible: en mi caso, Visual Studio Code.
- Navegador web actualizado: en mi caso, Google Chrome.

Pasos:

1. Guardar los archivos .js en la carpeta del proyecto.
2. Ejecutar el script en consola con Node (node DWEU_U03_A05_CP_E.js) o copiar y pegar el código directamente en la consola del navegador como en mi caso.
3. **Caso práctico 5:** comprobar que la función devuelve el array de etiquetas esperado.
4. **Ponte a prueba 5:** verificar que la expresión regular acepta los ejemplos válidos y rechaza los inválidos.
5. Revisar la consola (F12 → Consola) para ver los resultados y pruebas de autoría.

Objetivos de la práctica

- Probar y comprender el código teórico del apartado 5.
- Implementar extracción de etiquetas con expresiones regulares.
- Validar un documento de identidad ficticio con reglas estrictas.
- Documentar correctamente, **como siempre**, cada ejercicio con nombre, fecha y capturas de pantalla.
- Presentar el proyecto en formato PDF junto con los archivos fuente y pantallazos.

CONCLUSIÓN

Esta práctica ha permitido consolidar el uso de **expresiones regulares en JavaScript** para resolver problemas de filtrado y validación. Se ha comprobado cómo extraer etiquetas de cadenas con separadores diversos y cómo validar un formato de documento con precisión. La práctica nos ha sido muy útil para entender cómo poder aplicar regex en proyectos web para **controlar entradas y procesar texto de forma más clara**.